

Plan de projet expérimental — IFT-7026

Université Laval

Étudiants	Léopold Chappuis, Alexandre Eberhardt
Superviseur	Prof. Mohamed Aymen Saied
Directeur de programme	Prof. Pascal Tesson
Session	Session d'automne 2026

Sujet : Un ordonnanceur Linux utilisant un modèle d'IA avec une approche candidate-based au lieu d'algorithmes avec des heuristiques dans l'ordonnanceur classique (CFS et EEVDF).

1. Objectif du projet

L'objectif du projet est d'étudier comment un modèle d'IA pourrait remplacer les algorithmes utilisés par l'ordonnanceur Linux. Il faut se demander quel type de modèle est pertinent, comment l'interfacer avec le noyau Linux, puis étudier tous les compromis impliquant les performances (la taille du modèle, la précision, la latence, le type d'entraînement...), pour explorer les perspectives. L'objectif final est de proposer un draft d'article, pour le soumettre aux workshops de début d'année, afin de le faire publier dans une revue pertinente.

2. Résumé

Dans le cours GLO-7030, nous avons réalisé un projet consistant à imiter l'ordonnanceur Linux avec un modèle d'IA, nous avons obtenu des résultats intéressants (une précision de 73,22% sur le premier choix d'ordonnancement et une précision de 96,05% sur le top 3 des choix.). Nous souhaitons pousser plus loin ce projet, en étudiant en détail chaque possibilité, pour étudier l'influence de chaque paramètre, avec une méthodologie scientifique et reproductible, pouvant mener à un article scientifique.

Nous pensons que c'est intéressant et que cela présente un apport à l'état de l'art de la recherche, car cela est au croisement de l'IA, à travers le modèle de deep learning utilisé, et de l'informatique bas niveau, car nous traitons de l'ordonnanceur du noyau Linux.

3. Contenu et travaux à effectuer

Le projet serait structuré en trois phases.

Phase 0 : Consolidation du travail déjà effectué et définition du protocole expérimental

- Nettoyer le code existant et le simplifier pour permettre de le réutiliser de façon propre.
- Définition de l'algorithme comparé et utilisé en baseline (CFS/EEVDF).
- Mise en place de métriques claires, mesurables et reproductibles à utiliser dans la suite de notre travail.
- Définition des données à utiliser, pour l'entraînement et l'évaluation, pour des résultats objectifs et représentatifs des performances réelles.
- Revue de littérature et positionnement (restriction du sujet ou séparation en plusieurs sujets selon les perspectives).

Phase 1 : Contribution méthodologique

- Entraînement du modèle choisi avec les données choisies dans la phase 0.
- Évaluation en boucle fermée et comparaison avec la baseline.

- Amélioration itérative du modèle et de l'entraînement pour obtenir de meilleures performances.
- Expérimentation de méthodes pour obtenir des modèles plus légers, afin de réduire le temps d'inférence, et comparaison des performances (distillation, teacher-student...)

Phase 2 : Déploiement noyau réel

- Interfacement du modèle appris et choisi, en prenant en compte le temps d'inférence, avec un véritable noyau Linux, remplaçant l'ordonnanceur présent.
- Observer l'impact de la latence (lié à l'inférence) par rapport au temps de l'ordonnancement.
- Si possible, comparer les performances du noyau, avec le modèle ou l'ordonnanceur classique, pour une exacte même suite de tâches.

4. Modalités d'évaluation

Critère	Pondération
Présentation intermédiaire à la moitié de la session, pour présenter l'avancement et les perspectives pour la suite, avec le plan de l'article (peut être fait en visio).	30%
Présentation finale à la fin de la session, pour présenter les résultats finaux et le draft d'article réalisé. (peut être fait en visio).	30%
Le draft final, son apport à l'état de l'art et son taux de réponse aux attentes de ce projet.	30%
La méthodologie complète et la reproductibilité des résultats	10%

5. Échéancier

Jalon	Échéance
Présentation intermédiaire	fin octobre, (avant le 23 octobre, avant la semaine de lecture).
Présentation finale	mi décembre (avant le 11 décembre, fin de la session)
Le draft final, son apport à l'état de l'art et son taux de réponse aux attentes de ce projet. (30%)	avant le 1 ^{er} janvier
La méthodologie complète	avant le 1 ^{er} janvier

6. Livrables

- Le draft de l'article, finalisé et complet, relu par un spécialiste.
- La méthodologie complète
- Code propre complet permettant de reproduire tous les résultats et l'entraînement.

7. Références (préliminaires)

- [1] Jingde Chen, Subho S. Banerjee, Zbigniew T. Kalbarczyk et Ravishankar K. Iyer. *Machine learning for load balancing in the linux kernel*. Dans *Proceedings of the 11th ACM SIGOPS Asia-Pacific Workshop on Systems*, APSys '20, p. 67–74, New York, NY, USA, 2020. Association for Computing Machinery.
- [2] Anusha G. H., Minal K., B. P. Varsha, Geethishree Mishra, Meghana G. K. et Madhusudhan

- K. N. *ML engine for linux scheduler*. *Quest Journals, Journal of Software Engineering and Simulation*, vol. 9, n° 7, p. 20–29, 2023.
- [3] Sampanna Yashwant Kahu. *KernelOracle : Predicting the linux scheduler’s next move with deep learning*, 2025.
 - [4] Atul Negi et P. Kishore Kumar. *Applying machine learning techniques to improve linux process scheduling*. Dans *TENCON 2005 – 2005 IEEE Region 10 Conference*, p. 1–6, 2005.
 - [5] Stéphane Ross, Geoffrey J. Gordon et Drew Bagnell. *A reduction of imitation learning and structured prediction to no-regret online learning*. Dans *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 15 de *Proceedings of Machine Learning Research*, p. 627–635, 2011.
 - [6] Vasuki Shankar. *Machine learning for linux kernel optimization : Current trends and future directions*. *International Journal of Computer Sciences and Engineering*, vol. 13, p. 56–64, 03 2025.
 - [7] Ion Stoica et Hussein Abdel-Wahab. *Earliest Eligible Virtual Deadline First : A flexible and accurate mechanism for proportional share resource allocation*. Rapport technique TR-95-22, Old Dominion University, 1995.
 - [8] *Sched_ext : BPF extensible scheduler class*. Documentation du noyau Linux. <https://docs.kernel.org/scheduler/sched-ext.html>.